

Universidad De San Carlos De Guatemala
Facultad De Ingeniería
Escuela De Ciencias Y Sistemas
Introducción A La Programación Y Computación 2
Ing. Jaime Francisco Yuman Ramirez
Aux. Fernando José Vicente Velasquez

Actividad De Laboratorio 17/06
Arquitectura Multi-Nivel (N-Tier) Y Patrón Lógico De Software (MVC) En .NET

Samuel Ernesto Marín Higueros
Carnet: 202307732
Sección: P

Guatemala, Miércoles 17/06/2026

Contenido

Actividad de Laboratorio: Arquitectura Multi-Nivel	3
(N-Tier) y Patrón MVC en .NET	3
Parte 1: Fundamentación Teórica y Análisis Crítico.....	3
1. El Tránsito hacia los Sistemas Distribuidos y Multi-Capa.....	3
2. Desacoplamiento Lógico con el Patrón MVC.....	4
Parte 2: Modelado del Ciclo de Vida y Enrutamiento Semántico	5
1. Mapeo Analítico de URLs.....	5
2. Diagramación del Flujo Interactivo	5
Parte 4: Auditoría y Control de Calidad	6
1. Prueba de Cohesión (GET)	6
2. Evaluación de Antipatrones	6
Conclusión de la Auditoría.....	7
Parte 5: Referencias Bibliográficas.....	7

Actividad de Laboratorio: Arquitectura Multi-Nivel (N-Tier) y Patrón MVC en .NET

Parte 1: Fundamentación Teórica y Análisis Crítico

1. El Tránsito hacia los Sistemas Distribuidos y Multi-Capa

La Limitación del Monolito Local

Cuando la interfaz de usuario, la lógica de negocio y el almacenamiento de datos se encuentran en una sola máquina física, el sistema presenta problemas de escalabilidad y sincronización. Todos los usuarios dependen de un único equipo, lo que limita el crecimiento de la aplicación. Además, si la máquina falla, todo el sistema queda fuera de servicio. Este enfoque también dificulta el acceso concurrente y el mantenimiento de la información.

Distinción Crítica: Layers vs. Tiers

Las capas (Layers) representan una separación lógica dentro del código fuente de la aplicación. Su propósito es organizar responsabilidades y facilitar el mantenimiento.

Los niveles (Tiers) representan una separación física de los componentes en diferentes servidores o equipos. Cada nivel puede ejecutarse en hardware distinto y comunicarse a través de una red.

En resumen:

- Layers: organización lógica del software.
- Tiers: distribución física de la infraestructura.

Responsabilidades en la Arquitectura de 3 Niveles

Nivel 1: Capa de Presentación (Presentation Tier)

Su función es interactuar con el usuario, mostrar información y capturar datos de entrada. Generalmente utiliza tecnologías como HTML, CSS, JavaScript, Razor Views o navegadores web.

Nivel 2: Capa de Aplicación o Negocio (Application Tier)

Contiene las reglas de negocio, validaciones y procesamiento de información. Actúa como intermediario entre la presentación y los datos. En .NET suele implementarse mediante controladores, servicios y clases de negocio.

Nivel 3: Capa de Datos (Data Tier)

Es responsable del almacenamiento, recuperación y administración de la información. Generalmente utiliza sistemas gestores de bases de datos como SQL Server, MySQL o PostgreSQL.

Seguridad Perimetral

Exponer directamente una base de datos a Internet representa un riesgo crítico porque un atacante podría intentar acceder directamente a la información almacenada, ejecutar consultas maliciosas o comprometer la integridad de los datos.

La buena práctica consiste en mantener la base de datos en una red privada y permitir el acceso únicamente desde la capa de aplicación. De esta forma, el servidor de aplicaciones actúa como una barrera de seguridad entre los usuarios y la base de datos.

2. Desacoplamiento Lógico con el Patrón MVC

La Crisis del Código Espagueti

Cuando las consultas SQL, la lógica de negocio y la interfaz gráfica se mezclan en un mismo archivo, el código se vuelve difícil de mantener, depurar y ampliar. Además, las pruebas unitarias se complican porque las responsabilidades no están claramente separadas.

Este problema genera un alto acoplamiento y una baja mantenibilidad del software.

Separación de Preocupaciones (SoC)

- **Modelo (Model)**

Representa los datos y las entidades del sistema. Su responsabilidad es almacenar y gestionar la información. El modelo no debe conocer cómo se presentan los datos al usuario.

- **Vista (View)**

Es la encargada de mostrar la información al usuario. Se considera una entidad pasiva porque recibe datos y los presenta visualmente. No debe contener lógica de negocio ni consultas directas a bases de datos.

- **Controlador (Controller)**

Actúa como intermediario entre la vista y el modelo. Recibe las solicitudes del usuario, coordina el procesamiento de la información y decide qué vista debe mostrarse.

Métricas de Ingeniería de Software

El patrón MVC promueve una alta cohesión porque cada componente posee una responsabilidad específica y bien definida.

También favorece un bajo acoplamiento, ya que los componentes interactúan mediante interfaces claras y controladas, reduciendo las dependencias entre ellos.

Como resultado, las aplicaciones son más fáciles de mantener, probar, extender y reutilizar.

Parte 2: Modelado del Ciclo de Vida y Enrutamiento Semántico

1. Mapeo Analítico de URLs

URL Entrante del Cliente	Clase Controladora Buscada por el Framework	Método (Acción) Ejecutado	Parámetro id Inyectado
https://ingenieria.usac.edu.gt/ControlAcademico/Login	ControlAcademicoController	Login	Ninguno
https://ingenieria.usac.edu.gt/Estudiante/Historial/20260123	EstudianteController	Historial	20260123
https://ingenieria.usac.edu.gt/Asignacion/Detalle/10	AsignacionController	Detalle	10
https://ingenieria.usac.edu.gt/Home	HomeController	Index	(Ninguno / Opcional)

2. Diagramación del Flujo Interactivo

- *Paso 1*

El usuario interactúa con la aplicación web haciendo clic en un enlace o botón desde el navegador. Esto genera una solicitud HTTP hacia el servidor.

- *Paso 2*

El sistema de enrutamiento de ASP.NET Core analiza la URL recibida y determina qué controlador y qué acción deben ejecutarse según la plantilla de rutas configurada.

- *Paso 3*

El controlador correspondiente recibe la petición, valida la información y solicita los datos necesarios al modelo.

- *Paso 4*

El modelo proporciona los datos requeridos por la aplicación. El controlador recibe esos datos y los envía a la vista.

- *Paso 5*

La vista genera dinámicamente el código HTML utilizando la información recibida. El servidor devuelve ese HTML al navegador y el usuario visualiza la página renderizada.

Parte 4: Auditoría y Control de Calidad

1. Prueba de Cohesión (GET)

Se realizó una prueba accediendo a la ruta:

/Estudiante/Listar

La acción Listar() del controlador tiene como única responsabilidad recuperar la colección de estudiantes almacenada en memoria y enviarla a la vista mediante la instrucción:

```
return View(_baseDatosMemoria);
```

Durante la revisión se verificó que el método no contiene lógica matemática compleja, algoritmos de procesamiento, consultas SQL incrustadas ni operaciones ajenas a su propósito principal. Esto demuestra que el controlador actúa únicamente como intermediario entre el modelo y la vista, respetando el principio de Alta Cohesión.

La cohesión se refiere al grado en que los elementos de un módulo están relacionados con una única tarea. En este caso, la acción Listar() se enfoca exclusivamente en despachar información hacia la interfaz de usuario, lo que facilita el mantenimiento, la comprensión del código y la realización de pruebas futuras.

Además, la separación entre el modelo (Estudiante), el controlador (EstudianteController) y la vista (Listar.cshtml) permite que cada componente mantenga responsabilidades claramente definidas, alineándose con las buenas prácticas del patrón MVC.

2. Evaluación de Antipatrones

Se realizó una inspección del archivo EstudianteController.cs con el objetivo de identificar posibles antipatrones de diseño, especialmente el conocido como Fat Controller (Controlador Gordo).

Tras la revisión se determinó que ninguno de los métodos implementados supera las 20 líneas de código recomendadas por la actividad. Las acciones Listar() y Registrar() son breves, legibles y poseen responsabilidades específicas:

Listar() se encarga únicamente de obtener los datos y enviarlos a la vista.

Registrar() realiza una validación básica de entrada y registra un nuevo estudiante en la colección simulada.

No se encontraron bloques extensos de lógica de negocio, consultas a bases de datos, cálculos complejos ni procesos que debieran pertenecer a otras capas de la aplicación.

La implementación sigue el principio de Skinny Controllers, donde el controlador funciona como un coordinador de solicitudes y respuestas, delegando las responsabilidades específicas a otras capas cuando sea necesario. Este enfoque mejora la mantenibilidad, la reutilización del código y la escalabilidad del sistema.

Conclusión de la Auditoría

La aplicación desarrollada cumple satisfactoriamente con los criterios de desacoplamiento promovidos por la arquitectura MVC. Se observa una adecuada separación de responsabilidades entre los componentes, una alta cohesión interna y un bajo acoplamiento entre módulos.

Asimismo, el controlador mantiene un diseño limpio y compacto, evitando antipatrones comunes y facilitando futuras tareas de mantenimiento, pruebas unitarias y ampliación de funcionalidades.

Parte 5: Referencias Bibliográficas

- *Facultad de Ingeniería, USAC. (2026). Sesión 11: Modelado Base y Arquitecturas de Despliegue. Evolución de Sistemas Distribuidos, Fundamentos del Modelo Cliente-Servidor y Diseño Físico Multi-Capas (N-Tier). Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.*
- *Facultad de Ingeniería, USAC. (2026). Sesión 12: Arquitectura y Componentes del Patrón MVC. Desacoplamiento Lógico de Software, Ciclo de Vida de las Peticiones y Enrutamiento en Aplicaciones Interactivas Modernas. Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.*